Machine Learning & Deep Learning applications for environmental analysis and modeling

Module 1: Classical Machine Learning

20 May 2026 Mirko D'Andrea
mirko.dandrea@cimafoundation.org

Module 1: Classical Machine Learning

Date: 20 May 2026 **Instructor:** Mirko D'Andrea (mirko.dandrea@cimafoundation.org)

Implementation of classical ML algorithms with a focus on **fire risk assessment**.

- **Introduction to ML: fundamental concepts**
- **Classical ML algorithms**
- **Hands-on session:**
- Wildfire susceptibility using Random Forest

Module 2: Deep Learning

Date: 27 May 2026 **Instructor:** Luca Monaco (luca.monaco@cimafoundation.org)
Introduction to Deep Learning: Fundamental concepts.
The Modern ML Stack:

- **Data management:** Handling large-scale data with **Zarr**.
- **Model frameworks:** Simplifying workflows via **PyTorch Lightning**.
- **Configuration management:** Using **Hydra** for flexible experiment setups.
- **Experiment tracking:** Logging and versioning with **MLFlow**.

Hands-on applications:

- Precipitation forecast post-processing.
- Urban thermal comfort modeling.

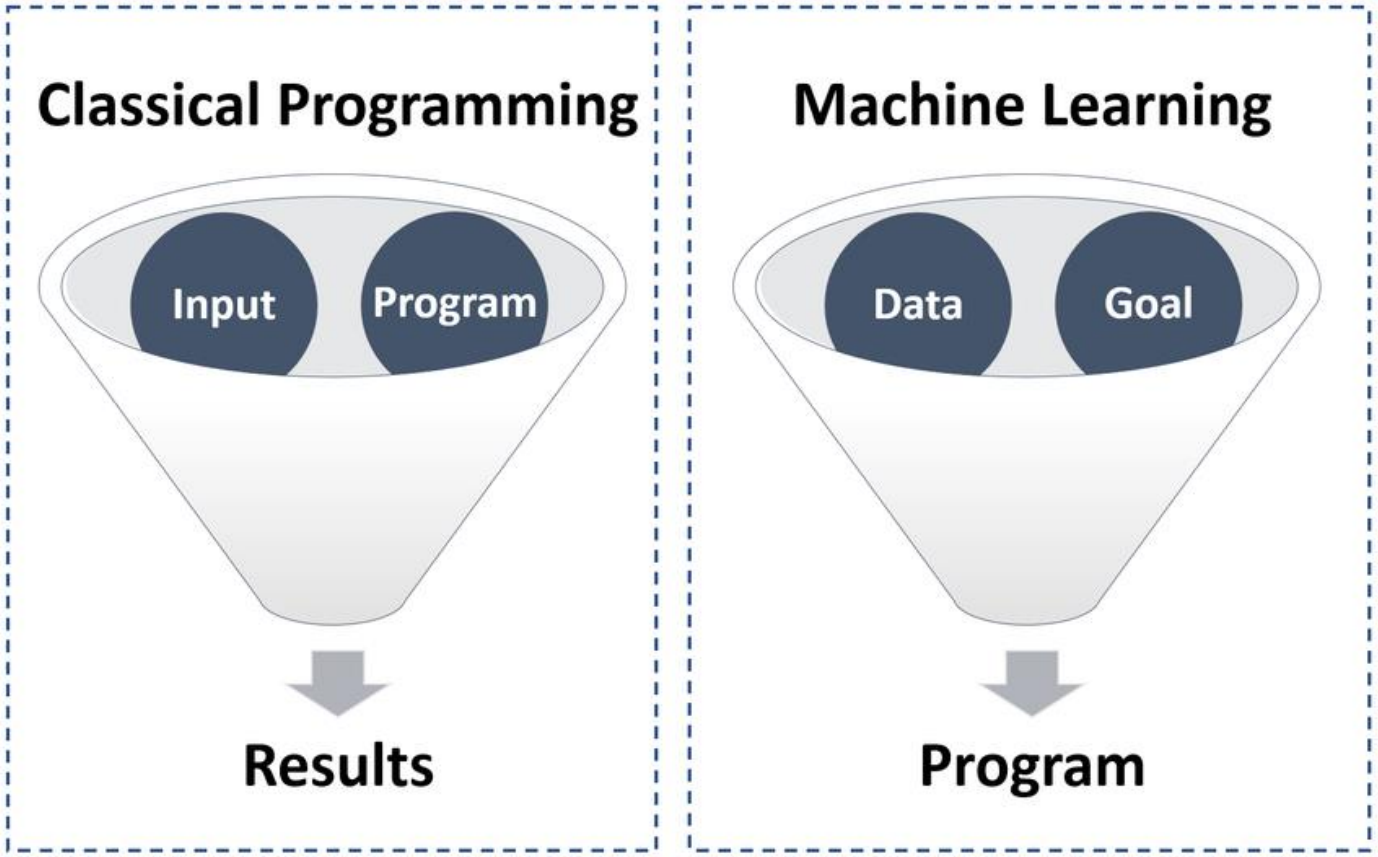
Final Assessment:

Evaluation for this course will be based on a **written report** detailing the application of the methods discussed. You must follow the entire course to be eligible for the final assessment.

Module 1 - Outline

- **Intro**
- Basic ML Concepts
- Models overview
- Wildfire susceptibility using Random Forest

What is Machine Learning?



- Machine learning (ML) is a type of computational technology that enables machines (i.e., computers) to learn and make predictions without being explicitly programmed.
- ML includes algorithms capable of learning from data, through the modeling of the hidden relationships between a set of input and output variables.

Machine Learning in environmental modeling

https://xkcd.com/1838/

ML performs particularly well to model environmental hazards, which naturally present **complex interactions** and **non-linear** behaviors.

For example: wildfire occurrence can depend simultaneously on vegetation, topography, climate, human accessibility, etc.

Traditional rule-based models struggle to represent these interactions. Machine Learning can leverage this environmental data abundance and learn patterns and relations from historical data.

The first rule of Machine Learning: Garbage In -> Garbage Out !

- No matter how sophisticated or advanced the algorithms or models used in data analysis, if the underlying data is flawed or inaccurate, the output will be equally flawed.
- Any analysis or processing performed on **flawed data** will likely yield **unreliable or inaccurate results.**

Outline

- Intro
- **Basic ML Concepts**
- Models overview
- Wildfire susceptibility using Random Forest

Machine Learning Process

TRAINING DATA

Collect and prepare the data

ALGORITHM

Choose the right algorithm

LEARNING

Train the algorithm using the data

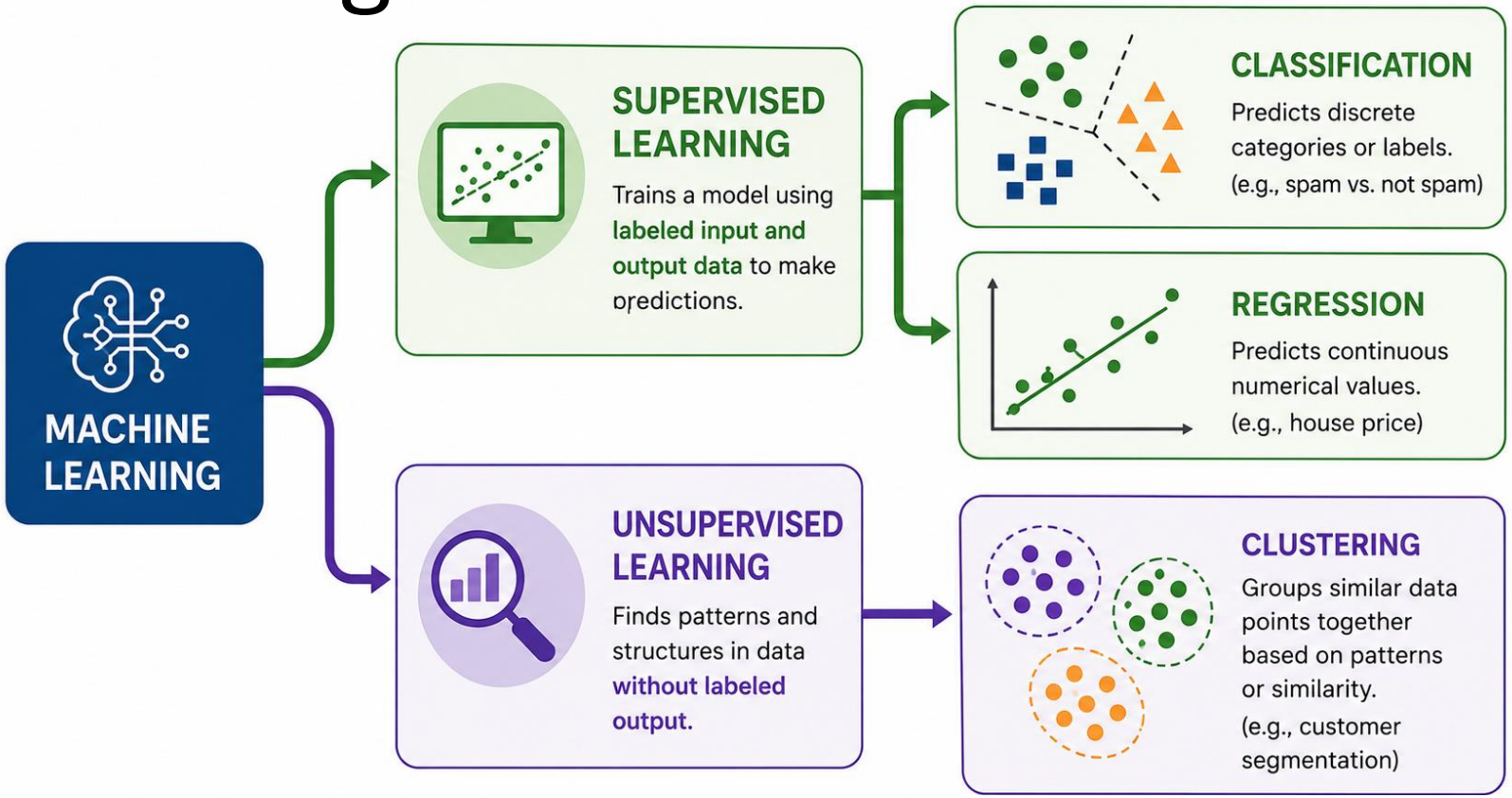
TRAINED MODEL

The model is ready to make predictions

RESULTS

Generate insights and make decisions

Machine Learning Tasks



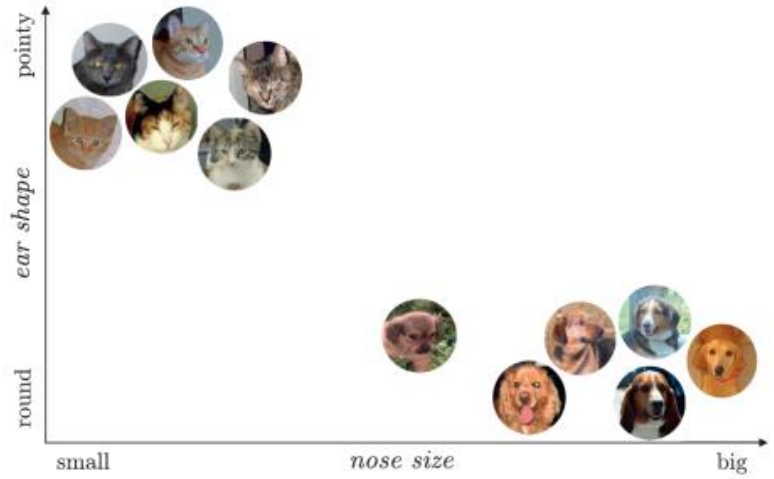
Supervised and unsupervised learning are the two main machine learning categories.

The key difference between supervised and unsupervised learning is the use of labels.

Features

input variables

A feature is an individual measurable property or characteristic of a phenomenon.



Labels

target variable

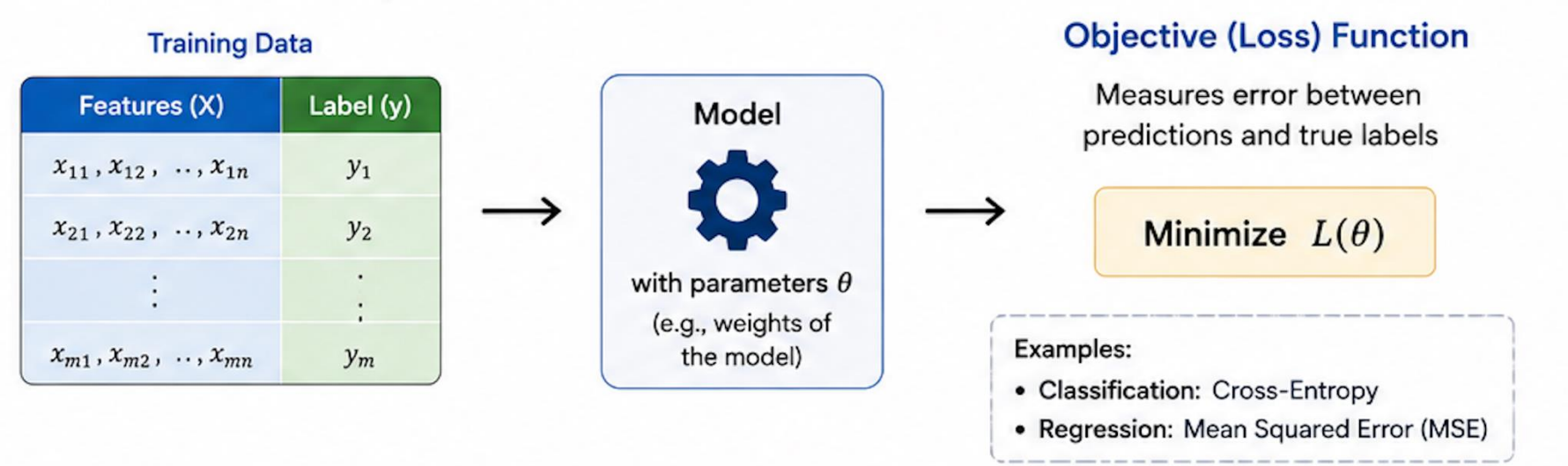
Our target variable, what we would like to predict using our ML algorithm.

	EAR SHAPE (FEATURE)	NOSE SIZE (FEATURE)	SPECIE (LABEL)
1	POINTY	SMALL	CAT
2	ROUND	MEDIUM	DOG

Both **features** and **labels** can be:

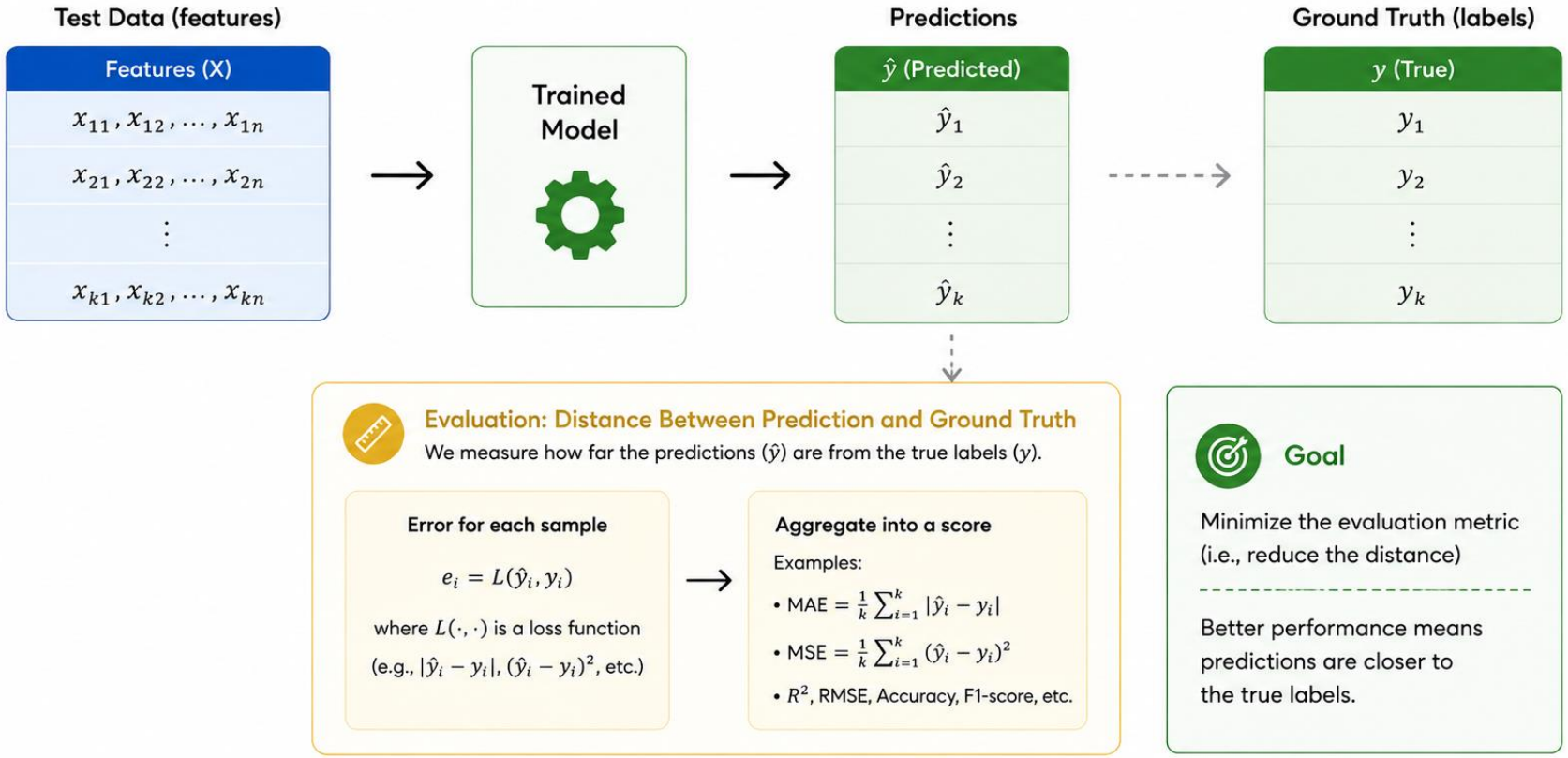
- **Numerical**: continuous values that can be measured on a scale (e.g. temperature)
- **Categorical**: discrete values that can be grouped into categories (e.g. vegetation type)

Training



The model learns the relationship between **features (X)** and the **label (y)** by minimizing an **objective (loss) function** on the training data, then its performance is evaluated on unseen test data.

Testing



Performance of the trained model should be tested against *unseen data*.

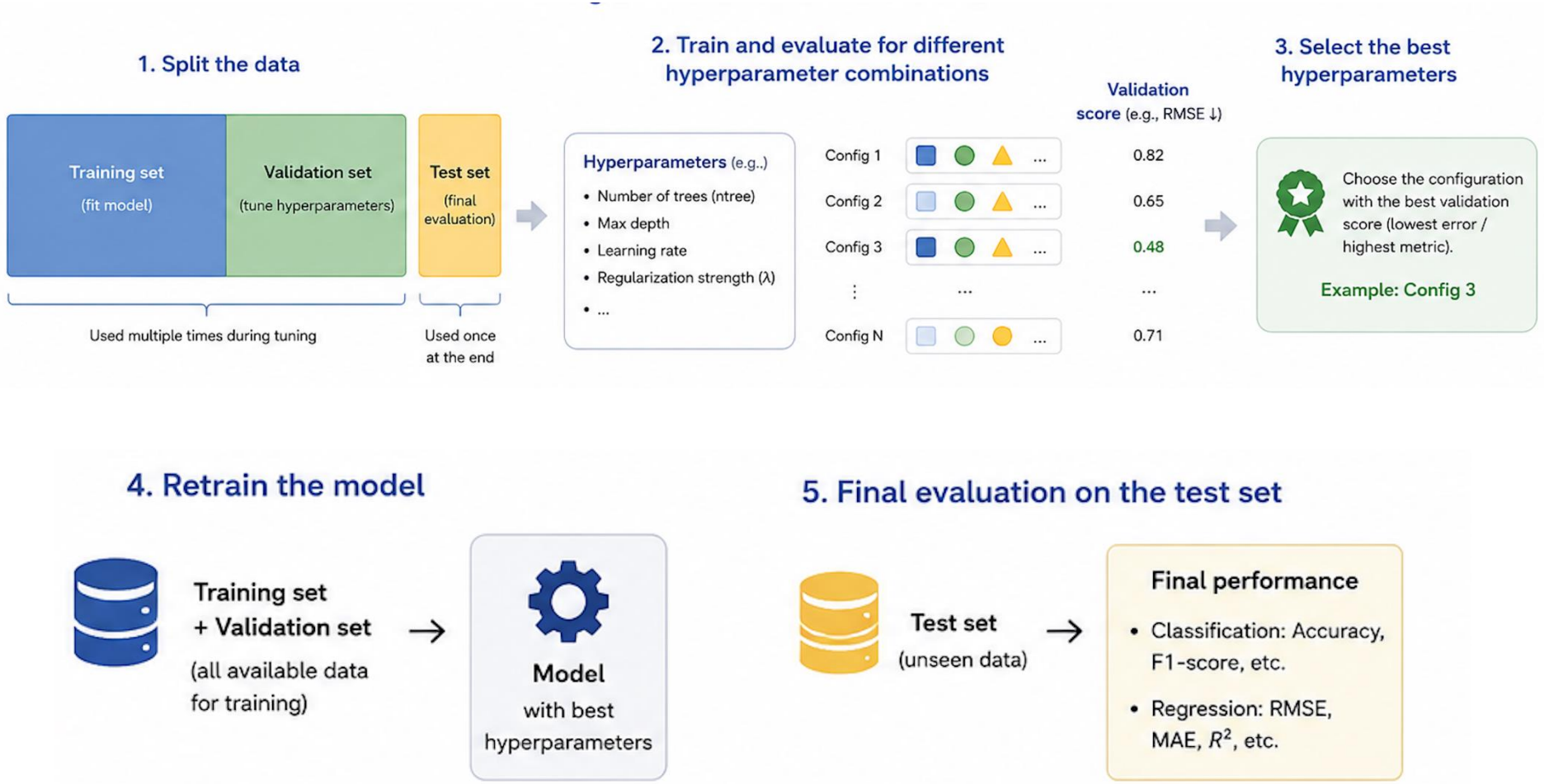
Examples for Environmental Modelling:

- Held-out environmental observations not used during training
- Tests generalization across new locations, seasons, or events
- Captures real-world variability and unseen environmental conditions

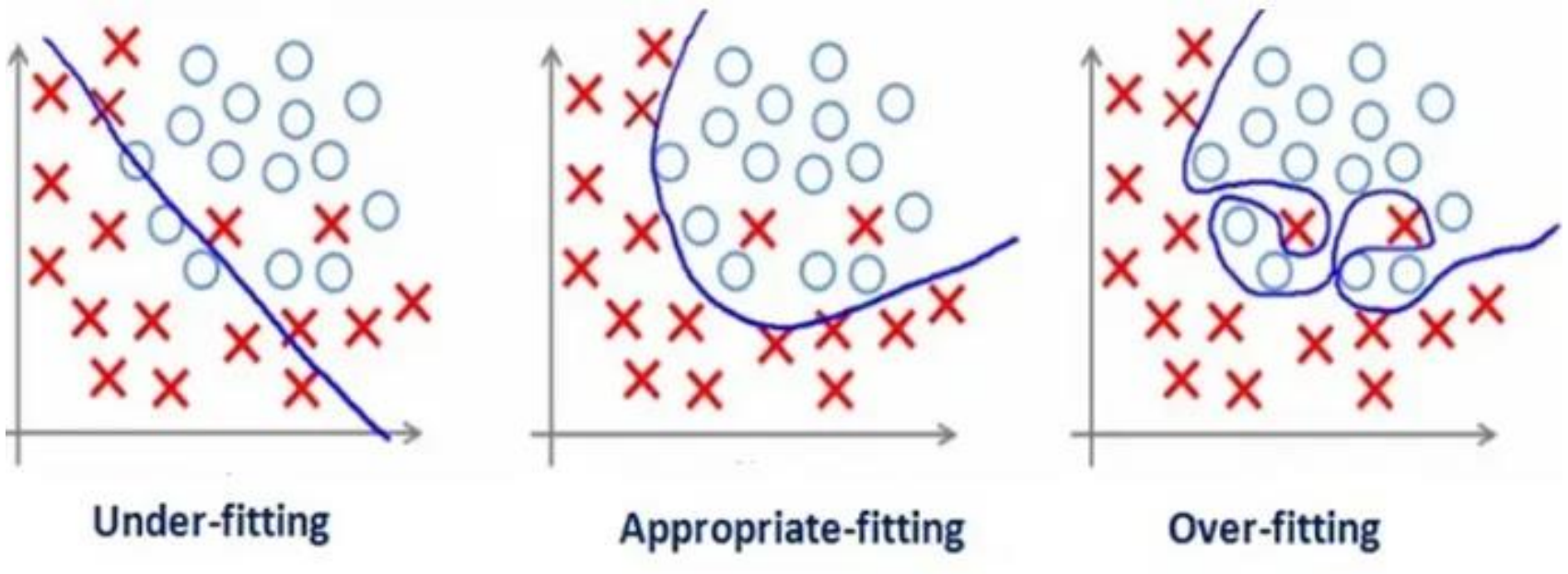
Model tuning

Models have some tunable **hyperparameters** that can influence the outcome of the training process

A subset of the training set called validation set is used to evaluate different hyperparameter values and select the model that generalizes best to unseen data.



Over and under fitting



The bias variance curse

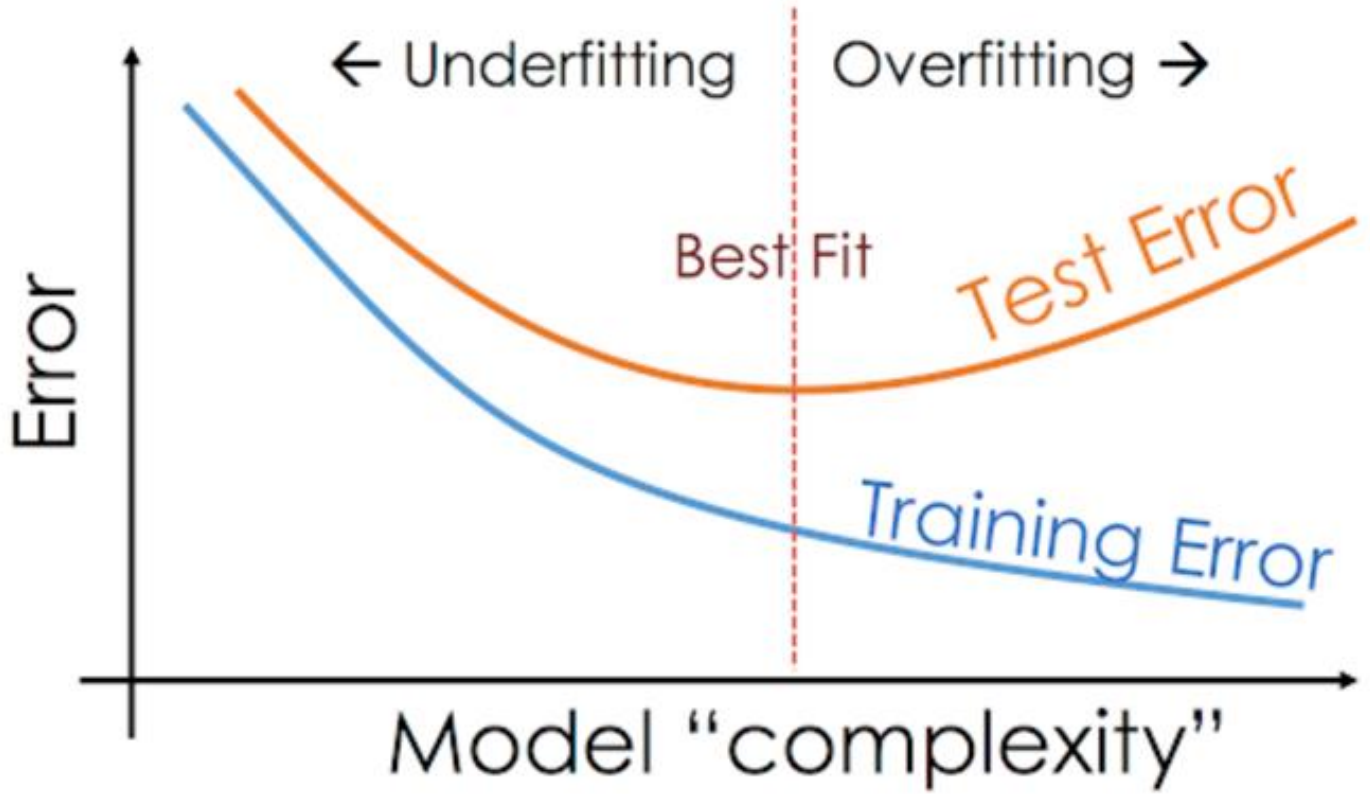
- **Bias** = *systematic mistake*

Variance = *instability*

As the complexity of model increases the bias decreases but the variance increases.

There is a trade-off between **bias** and **variance**.

You can't get both low bias and low variance at the same time.

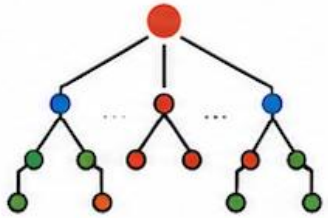


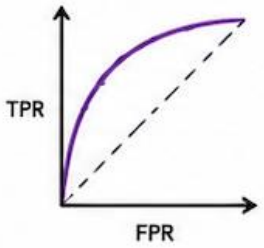
Typical ML workflow

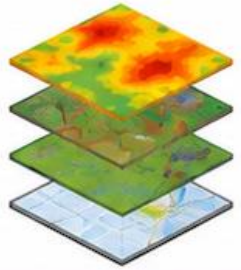
Data preparation

Model development

1 DATA COLLECTION → 2 DATA CLEANING → 3 FEATURE ENGINEERING → 4 TRAIN / VALIDATION / TEST SPLIT → 5 MODEL TRAINING → 6 MODEL EVALUATION → 7 INTERPRETATION → 8 PREDICTION / DEPLOYMENT







Gather raw data from various sources Example: Satellite data, DEM, roads etc.	Handle missing values and outliers Correct errors and unify formats and spatial resolution	Create meaningful predictors from raw data Example: slope, aspect, distance to roads, NDVI	Split data to train the model and evaluate it Avoid spatial and temporal leakage	Learn relationships between predictors and target variable Example: Random Forest	Assess predictive performance Metrics: Accuracy, ROC-AUC, Confusion Matrix, etc.	Understand which variables drive predictions Feature importance, Partial Dependence, SHAP values	Apply the model to new areas or future scenarios Generate maps and inform decisions

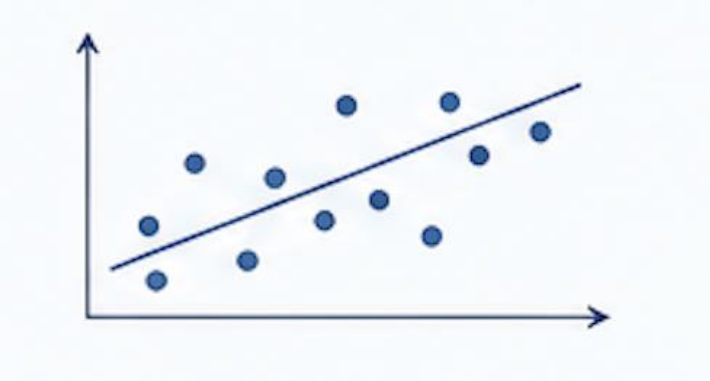
MULTIPLE ITERATIONS

Outline

- Intro
- Basic ML Concepts
- **Models overview**
- Wildfire susceptibility using Random Forest

Linear & Logistic Regression

Learn simple relationships between predictors and target variables.



Typical environmental applications

- Temperature trends
- Air quality estimation
- Binary hazard occurrence

Good baseline models and useful for explainable analyses

Strengths

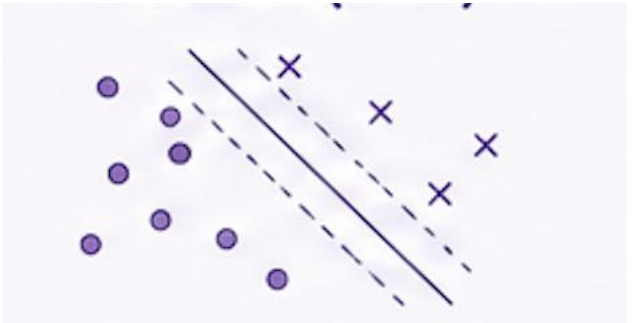
- Simple
- Fast
- Highly interpretable

Limitations

- Limited ability to model complex non-linear processes

Support Vector Machine (SVM)

Separates classes by finding the optimal decision boundary.



Typical environmental applications

- Land-cover classification
- Remote sensing
- Image segmentation

Widely used in remote sensing and geospatial classification problems.

Strengths

- Effective with limited datasets
- Handles complex class separation

Limitations

- Computationally expensive on large datasets
- Less interpretable

Gradient Boosting

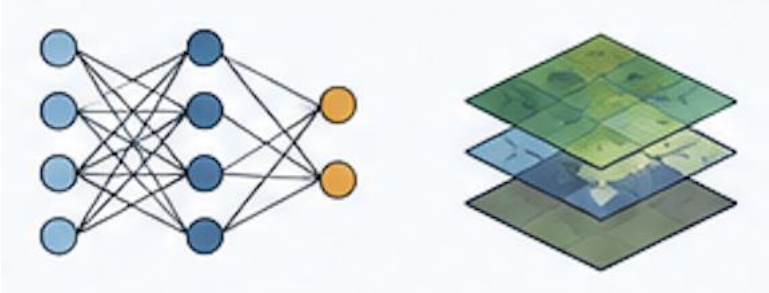
Multiple weak learners are built sequentially, each focusing on correcting mistakes done by previous ones.

Strengths
- Very high predictive performance

Limitations
- More difficult to interpret
- Requires hyperparameter tuning

Typical environmental applications
- Extreme-event prediction
- Risk mapping
- Environmental forecasting

High predictive performances and flexibility



Neural Networks & Deep Learning

Learn complex hierarchical patterns from large datasets.

Strengths
- Excellent for images and spatial data
- Can model highly complex relationships

Limitations
- Requires large datasets and computation
- Often behaves like a “black box”

Typical environmental applications
- Satellite image analysis
- Fire and smoke detection, flood mapping, etc.
- Weather prediction

Particularly powerful for computer vision and Earth observation applications.

Decision Trees

A Decision Tree splits the dataset into smaller and more homogeneous groups using sequential rules on predictor variables.

How it works

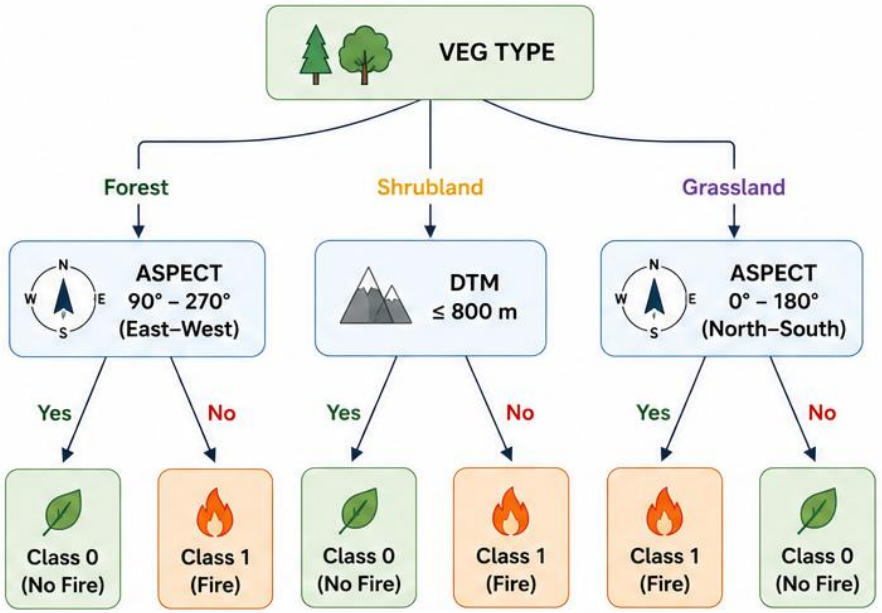
1. Start from all observations
2. Select the variable that best separates classes
3. Split the data into branches
4. Repeat recursively until terminal nodes are reached

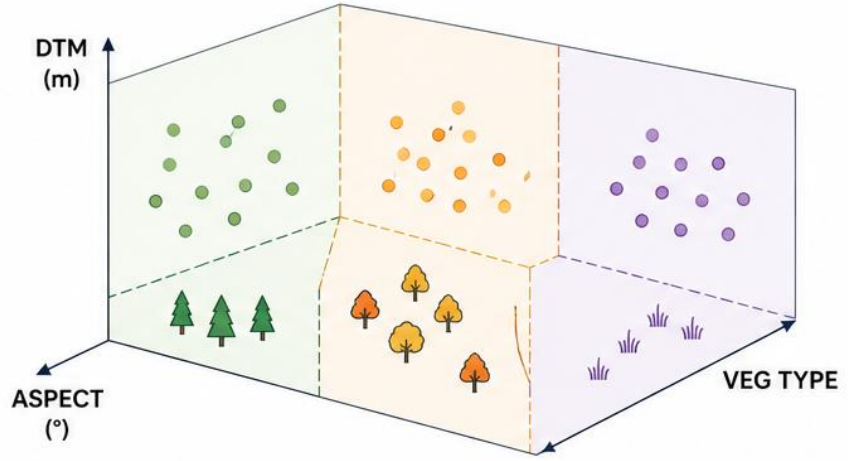
Advantages

- Simple and interpretable
- Handles numerical and categorical variables
- Captures non-linear relationships

Main limitation

Single trees are unstable and prone to overfitting.





Random Forest

Random Forest is an ensemble-learning algorithm that combines predictions from many independent decision trees.

How it improves Decision Trees

Each tree is trained on:

- a random subset of observations
- a random subset of predictor variables

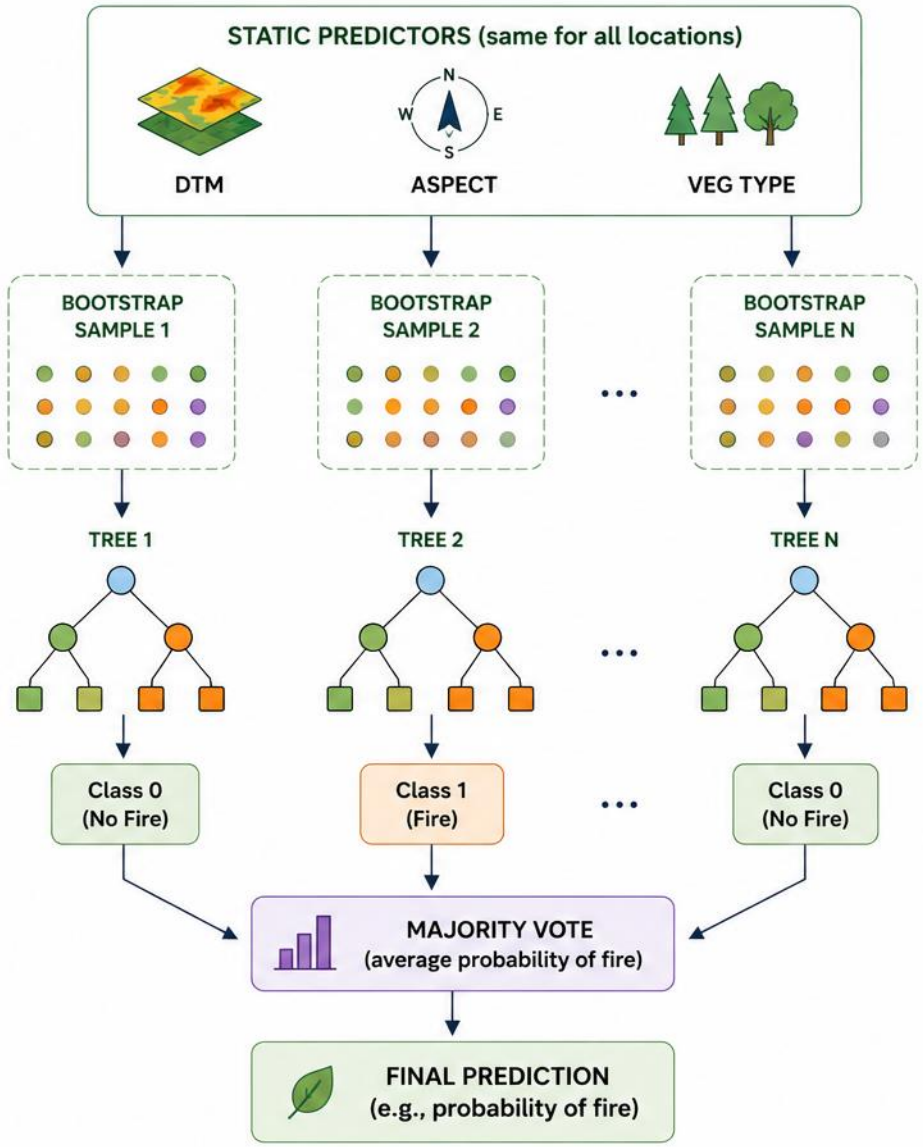
This increases model diversity and reduces overfitting.

Final prediction

Each tree produces a prediction:

- classification → majority vote
- probability → average probability across trees

"Random Forest trades some interpretability for better generalization and predictive robustness."



ntree

Number of generated trees

More trees → more stable predictions (until convergence).

mtry

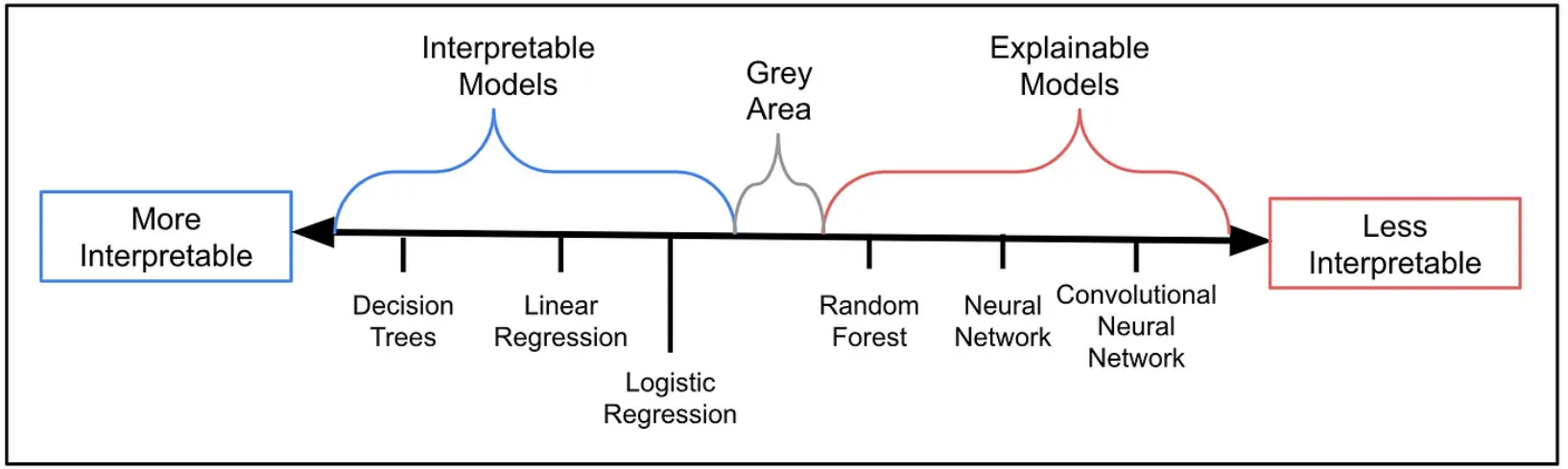
Number of variables randomly sampled at each split

Controls the diversity of trees and their correlation.

Models Interpretability

Simple models allow understanding how predictions are made, without extra tools.

Complex models often work like "***black boxes***". Additional methods are needed to understand how they make predictions.



https://miro.medium.com/v2/resize:fit:1400/format:webp/1*6-QWLr9obdDlUmVLtDXPrQ.png

Explainability focuses on understanding what happens inside a model, from input to output.

Outline

- Intro
- Basic ML Concepts
- Models overview
- **Wildfire susceptibility using Random Forest**

ML In practice: Random Forest for Forest Fire Susceptibility

geosciences

MDPI

Article

A Machine Learning-Based Approach for Wildfire Susceptibility Mapping. The Case Study of the Liguria Region in Italy

Marj Tonini [1,*], Mirko D'Andrea [2], Guido Biondi [2], Silvia Degli Esposti [2], Andrea Trucchia [2] and Paolo Fiorucci [2]

geosciences

MDPI

Article

Machine-Learning Applications in Geosciences: Comparison of Different Algorithms and Vegetation Classes' Importance Ranking in Wildfire Susceptibility

Andrea Trucchia [1,*], Hamed Izadgoshasb [1], Sara Isnardi [2], Paolo Fiorucci [1] and Marj Tonini [3]

CSIRO PUBLISHING

RESEARCH PAPER

https://doi.org/10.1071/WF22138

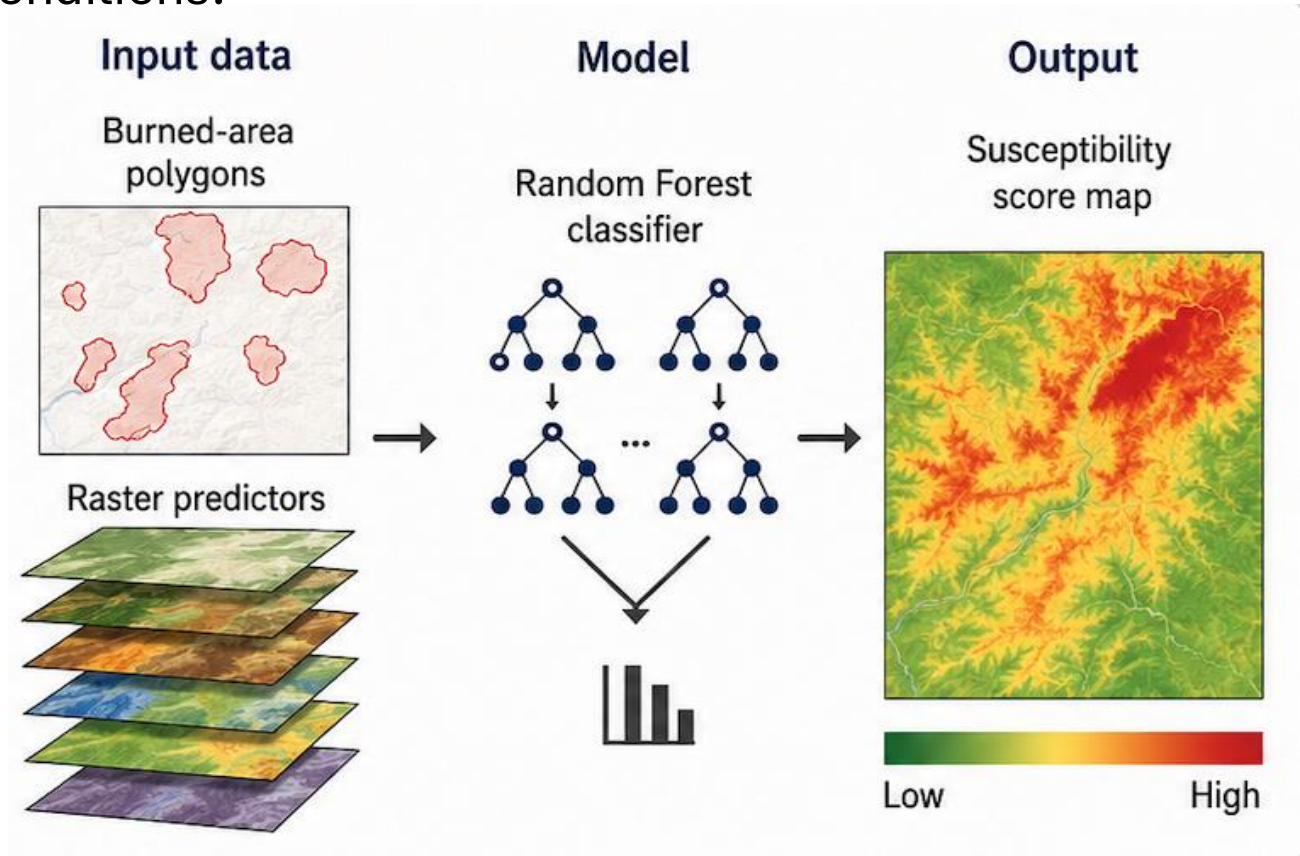
International Journal of WILDLAND FIRE

International Association of Wildland Fire

Wildfire hazard mapping in the eastern Mediterranean landscape

Andrea Trucchia[A,*], Giorgio Meschi[A], Paolo Fiorucci[A], Antonello Provenzale[B], Marj Tonini[C] and Umberto Pernice[A,D]

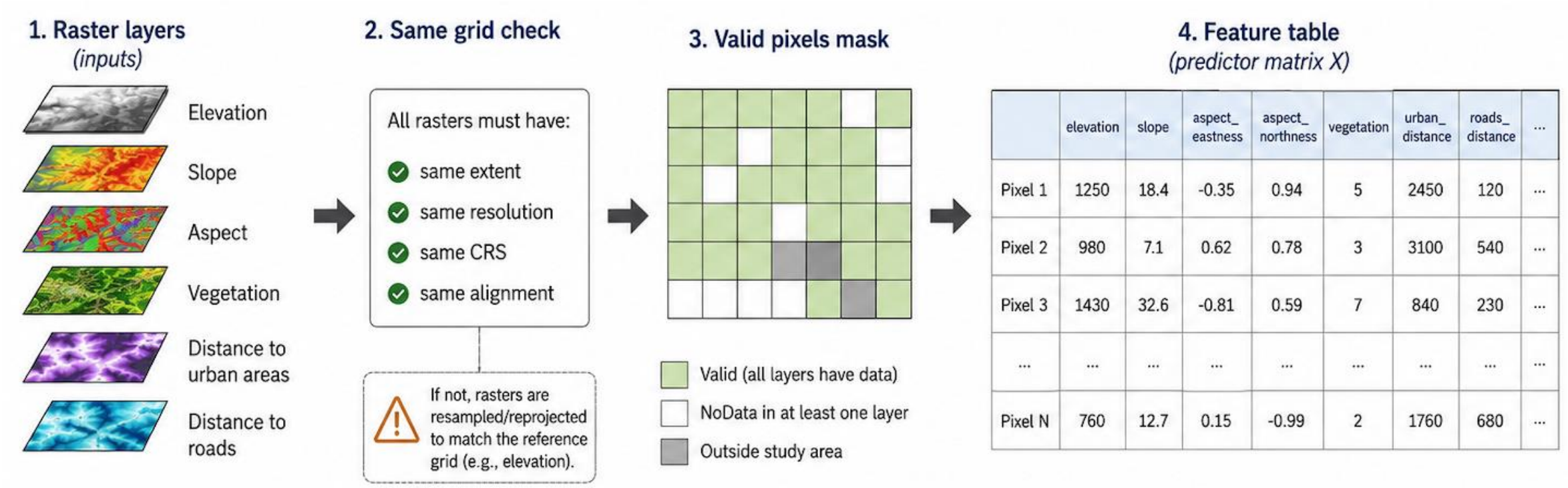
Goal: estimate the **relative probability** of wildfire occurrence as a proxy of wildfire susceptibility. Use occurred events, environmental and anthropogenic conditions.



Course material
https://github.com/mirkodandrea/wildfire-susceptibility-ml-lesson

Open in Colab

Raster Predictors and Feature Engineering



	elevation	slope	aspect_ eastness	aspect_ northness	vegetation	urban_ distance	roads_ distance	...
Pixel 1	1250	18.4	-0.35	0.94	5	2450	120	...
Pixel 2	980	7.1	0.62	0.78	3	3100	540	...
Pixel 3	1430	32.6	-0.81	0.59	7	840	230	...
...	...	...	...	...	...	...	...	...
Pixel N	760	12.7	0.15	-0.99	2	1760	680	...

The model uses environmental and anthropogenic predictors derived from raster layers.
All rasters are aligned to the same grid and stacked to build the feature table.

Categorical Variables

Some Random Forest implementations need numeric inputs, so categorical variables must be encoded first.

- **Ordinal encoding:** use when categories have order, e.g. low / medium / high.
- **One-hot encoding:** use when categories have no order, e.g. forest type or land-use class. Replace the categorical variable with 0/1 columns.
- **Target encoding:** use label/target relation to variable to sort categories. Can introduce overfit.

For this example, we will use One Hot Encoding for the **vegetation** categorical variable

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

Feature engineering, in code

```python
# Aspect: degrees → two orthogonal components
def add_aspect_features(frame):
  rad = np.deg2rad(frame["aspect"])
  frame["aspect_eastness"]  = np.sin(rad)
  frame["aspect_northness"] = np.cos(rad)
  return frame

# Vegetation: unordered codes → one binary column per class
def one_hot_encode_vegetation(frame):
  dummies = pd.get_dummies(
    frame["vegetation"].astype(int),
    prefix="vegetation", dtype=int,
  )
  return dummies.reindex(
    columns=VEGETATION_FEATURE_COLUMNS, fill_value=0,
  ).reset_index(drop=True)
```

Why these transforms?

- **Aspect is circular** — 1° and 359° point nearly the same direction. Sin/cos preserve direction with no discontinuity.
- **Vegetation is unordered** — code 3111 is not "between" 312 and 3112. One-hot encoding lets the model learn per-class effects.
- Result: **18-column feature matrix** ready for the Random Forest.

Burned vs unburned: what the features show

Class-wise medians (continuous predictors)

- Elevation: burned 467 m vs unburned 565 m — fires concentrate lower in the landscape
- Slope: 21.3° vs 18.8° — burned pixels sit on steeper terrain
- Aspect northness: −0.56 vs 0.00 — burned pixels skew strongly south-facing
- Aspect eastness: 0.16 vs 0.00 — a small eastward tilt
- Distance to urban / roads: essentially identical between classes — accessibility alone does not separate burned from unburned in this region

Takeaway: terrain (elevation, slope, aspect) carries clear discriminative signal; distance features carry almost none.

Vegetation: which classes burn?

Strongly over-represented in burned pixels

- Mediterranean evergreen scrub: 18.7% of burned vs 1.7% of unburned (~11× enrichment)
- Shrubland and scrub: 13.3% vs 2.8% (~5×)
- Warm-climate mixed forest: 21.0% vs 15.9% (moderate)

Under-represented in burned pixels

- Chestnut forest: 10.7% of burned vs 26.7% of unburned
- Moist-climate mixed forest: 4.5% vs 12.4%

One-hot encoding produces 12 vegetation columns, bringing the feature matrix to 18 total. The model can now learn class-specific fire propensity rather than assuming a numeric ordering.

What the EDA tells us before modelling

The features carry a real signal

- Terrain separates classes: lower, steeper, south-facing pixels are more likely to have burned
- Vegetation is the strongest single discriminator: scrub and Mediterranean classes burn far above their base rate

But no single feature is enough

- Distributions overlap heavily for each individual predictor
- A model needs to combine terrain **and** vegetation rather than threshold any one feature

Caveats to keep in mind

- Contrasts are vs sampled pseudo-absences, not the true landscape
- Spatial autocorrelation means nearby pixels are not independent — random splits will look optimistic

Pseudo-absences and Class Imbalance

Pseudo-absences are necessary when you have p**resence-only data**, meaning:

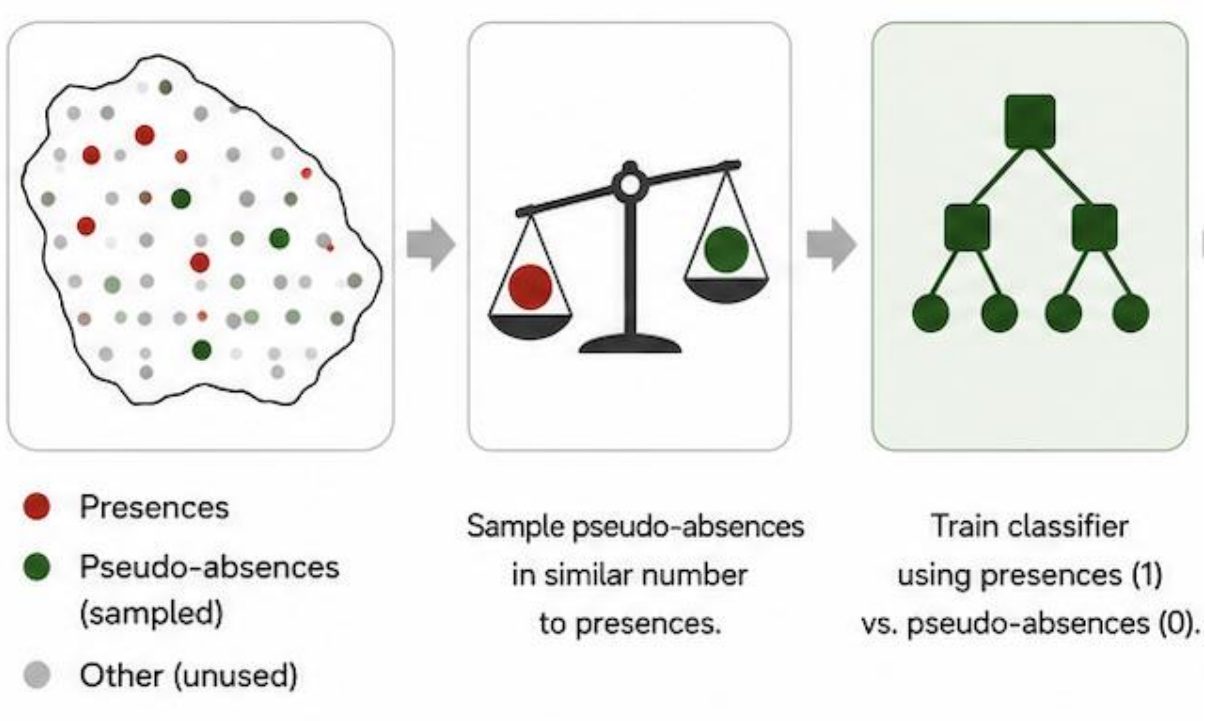
- we know where the event occurred
- but **true absences** are unknown

Treating all unknown areas as absences would lead to:

- Severe class imbalance
- Bias toward easy background areas
- Poor model generalization

Pseudo-absences are randomly generated from locations where the event is assumed not to occur.

Pseudo-absences are often sampled in similar number to presences to reduce class imbalance (50% presences, 50% pseudo absences)



Pseudo-absences provide a practical way to represent the absence class when true absences are unknown

Pseudo-absences, in code

```
# Why pseudo-absences? Fire is rare (1–5% of pixels).
# A 1:1 burned : sampled-unburned ratio prevents the
# model from learning the trivial 'predict unburned' rule.

burned = train_period_df.loc[train_period_df["target"] == 1]
unburned = train_period_df.loc[train_period_df["target"] ==
0].sample(
    n=len(burned),
    random_state=RANDOM_STATE,
)

# Shuffle so burned and unburned are interleaved.
train_pool = pd.concat([burned, unburned], ignore_index=True)\
    .sample(frac=1, random_state=RANDOM_STATE)\
    .reset_index(drop=True)

# Hold-out: KEEP real imbalance — do not balance.
test_df = add_period_labels(features, test_burned_mask, ...)
```

Balance for training, not for testing

- **1:1 ratio** — forces the model to learn discriminative features, not the base rate.
- **Result** — scores are a relative ranking, not calibrated probabilities.
- **Hold-out stays imbalanced** — 5,806 burned out of 398,697 pixels (1.5%) — mirrors operational reality.

Random Forest Hyperparameter Tuning

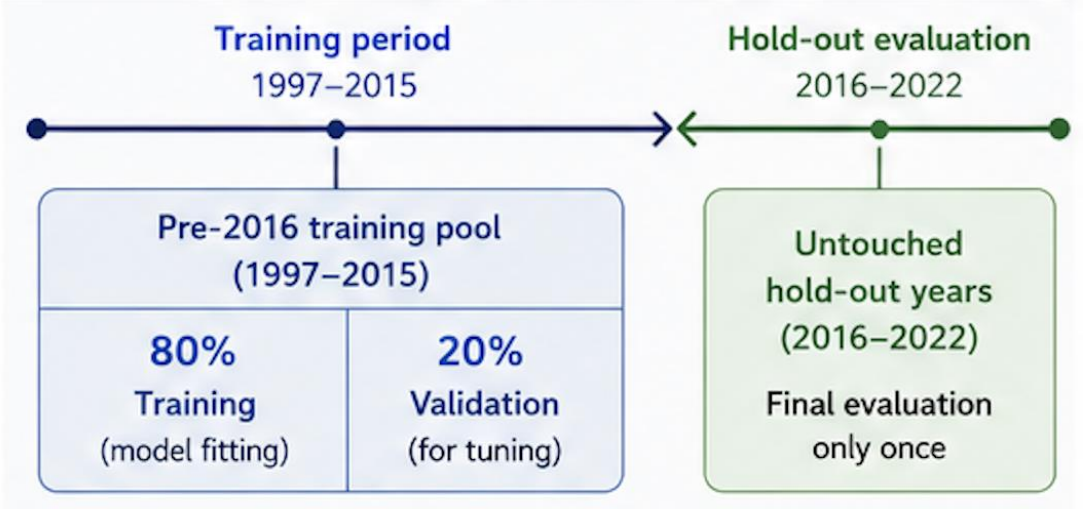
Why hyperparameter tuning matters

Random Forest performance strongly depends on the choice of hyperparameters.

Hyperparameter tuning aims to identify the model configuration that generalizes best to unseen data, avoiding both **Underfitting** and **Overfitting**

The tuning process is performed on a **validation dataset**, separate from the final hold-out test period.

Validation strategy used in this workflow: **temporal split**



n_estimators Number of trees	• More trees improve stability and robustness • Cost increases with number of trees • Performance often converges after enough trees
max_features Predictors at each split	• Smaller values increase tree diversity • Larger values may increase correlation between trees • Best value: sqrt → higher diversity, less overfitting
min_samples_leaf Minimum in leaf	• Smaller values → more detailed trees • Larger values → smoother, more generalized • Best value: 1 → fine local partitioning improved ranking
max_depth Tree depth limit	• Shallow trees reduce complexity • Deep trees capture more interactions • Best value: None → ensemble handled deep trees well

Best configuration selected

Final tuned model	
n_estimators	300
max_features	sqrt
min_samples_leaf	1
max_depth	None

Validation performance	
ROC-AUC	PR-AUC
≈ 0.84	≈ 0.84

After selecting the best hyperparameters:

1. Retrain the model on the full pre-2016 training pool
2. Evaluate once on the untouched 2016–2022 hold-out period

Hyperparameter tuning, in code

```
param_grid = {
    "n_estimators":     [150, 300],
    "max_features":     ["sqrt", 0.5],
    "min_samples_leaf": [1, 3, 5],
    "max_depth":        [None, 15],
}

# Reuse the fixed validation split — no k-fold reshuffle.
# Nearby pixels share terrain + fire history → k-fold leaks.
fold = [-1]*len(X_train) + [0]*len(X_valid)
split = PredefinedSplit(test_fold=fold)

search = GridSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    param_grid=param_grid,
    scoring="roc_auc", cv=split,
)
search.fit(X_tune, y_tune)
```

Grid search with a fixed fold

- **24 combinations** evaluated against ROC-AUC on the held-out validation rows.
- **PredefinedSplit** — we are using a predefined split for simplicity. We should consider spatial autocorrelation. The results could be inflated.
- **Winner** — 300 trees, sqrt features, no depth cap. Validation AUC 0.842.

Training the final model

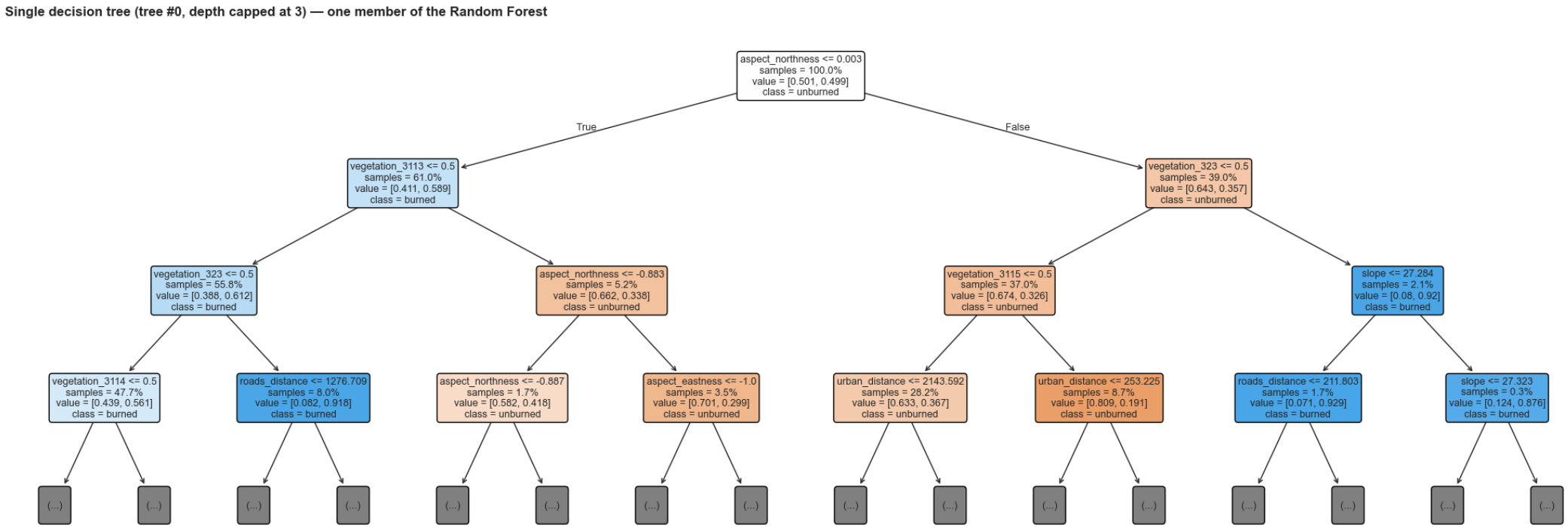
Two models, two purposes

- **validation_model** — fit on the 80% training split only, so the validation rows stay untouched and give honest tuning metrics.
- **rf_model** — refit on the full balanced 1997–2015 pool; used for the 2016–2022 hold-out and the susceptibility map.

Selected hyperparameters

- n_estimators=300, max_features="sqrt", min_samples_leaf=1, max_depth=None.

Peeking inside one tree



- Root split is **aspect_northness** — south-facing slopes (left branch) skew "burned"; north-facing (right) skew "unburned".
- Vegetation one-hots dominate deeper splits — confirms the EDA signal. One tree is noisy; the forest averages 300 of these for a stable score.

Evaluation metrics

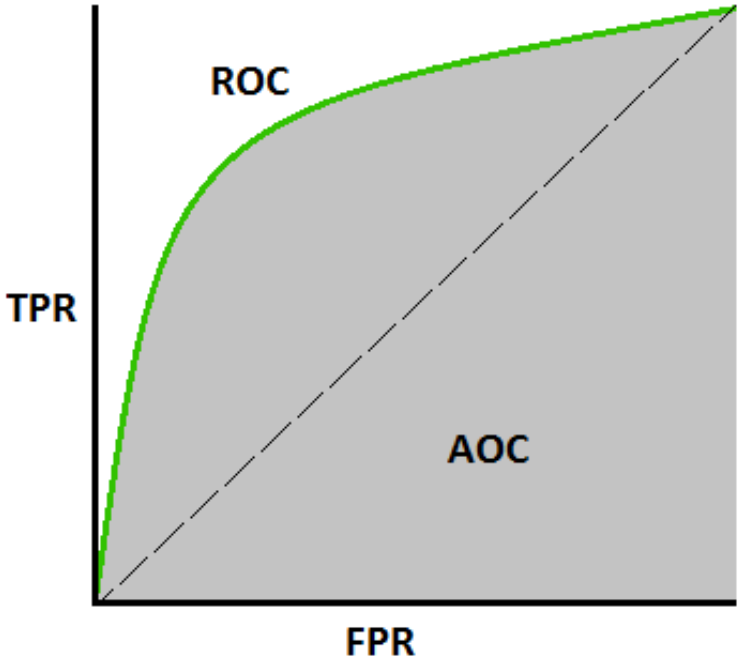
ROC-AUC — threshold-free ranking quality

- P(random burned scores > random unburned). 1.0 = perfect, 0.5 = chance.
- Plots TPR vs FPR across all thresholds — no cutoff needed.

Recall@TopK% — operational metric

- If we prioritize the top K% of the landscape by score, what share of future fires does that zone capture?
- Speaks directly to budget-constrained decisions.

⚠ *Accuracy is unsafe under 1.5% prevalence; ranking metrics are the right tool for susceptibility.*



ROC curve + score distributions, in code

```
# Score the hold-out
scores = rf_model.predict_proba(X_test)[:, 1]

# ROC curve from estimator
fig, ax = plt.subplots(figsize=(6, 4.5))
RocCurveDisplay.from_estimator(
    rf_model, X_test, y_test, name="Random Forest", ax=ax)
ax.plot([0, 1], [0, 1], linestyle="--")

# Score histograms by class
for target, color in [(0, "#2563eb"),
                      (1, "#dc2626")]:
    ax.hist(scores[y_test == target],
            bins=30, alpha=0.5, color=color)

# Hold-out ROC-AUC = 0.798
```

Two diagnostic plots

- ROC curve: how well burned pixels rank above unburned ones across every threshold.
- Score histograms: where each class lives along the susceptibility axis.

What to look for

- ROC curve well above the diagonal — the model adds ranking value over chance.
- Burned distribution shifted right of unburned — separation, not perfect overlap.

Recall@Top%, in code

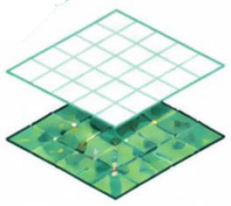
```python
# Operational question: if we map only the top K% of the
# landscape as high-susceptibility, what share of future
# burns falls inside that zone?

def recall_at_top_percent(y_true, score, top_share):
    threshold = np.quantile(score, 1 - top_share)
    return recall_score(y_true, score >= threshold)

# Score every pixel in the 2016–2022 hold-out (398,697 rows).
test_scores = rf_model.predict_proba(X_test)[:, 1]

for  k in [0.10, 0.25, 0.50]:
    r = recall_at_top_percent(y_test, test_scores, k)
    print(f"Top {k:.0%} → {r:.1%} of burns captured")

# Top 10% → 49.5%  Top 25% → 69.8%  Top 50% → 86.5%
# Hold-out ROC-AUC = 0.798
```

Rank, don't classify

- **Top 10% of the map** captures roughly half of 2016–2022 burned pixels — a 5× lift over the landscape base rate.
- **Percentile thresholds** are portable across runs; raw score cutoffs are not.
- **ROC-AUC 0.798** — a useful ranker, not a perfect predictor. Weather and ignition aren't in the model.

Environmental ML Challanges

Environment and geospatial data have unique characteristics that impact how we build, evaluate and apply ML models.
Good ML in environmental applications requires **spatial**, **temporal** and **domain awareness**.



SPATIAL AUTOCORRELATION

Nearby locations are more similar

IMBALANCED DATA

Events (e.g., fires, floods) are rare -> bias
Careful sampling and metrics are required

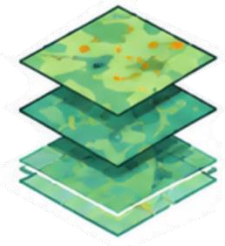
MISSING AND NOISY DATA

Gaps due to clouds, sensor limits etc.

Requires robust preprocessing

TEMPORAL NON-STATIONARITY

Relationships change over time (seasons, land use, climate)



HETEROGENEOUS DATA SOURCES

Different resolutions, formats and scales
Combining in-situ, remote sensing and derived data
Requires harmonization and feature engineering

INTERPRETABILITY & DECISION-MAKING

Models must be trustworthy and explainable
Stakeholders need clear and actionable outputs